

MODUL PRAKTIK/PRAKTIKUM

IF451 – Advance Web Programming

PROGRAM SARJANA S1

FAKULTAS TEKNIK DAN INFORMATIKA

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA

Gedung B Lantai 5, Kampus UMN

Jl. Scientia Boulevard, Gading Serpong, Tangerang-15811 Indonesia

Telp: +62-21.5422.0808, web: <http://www.umn.ac.id>



Modul Week 12 : File Upload & Basic Authentication

Pada praktikum ini, kita akan membuat sebuah website dengan fitur file upload dan authentication:

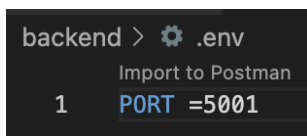
- User : User terbagi menjadi 2 role : admin dan mahasiswa. User dapat di create hanya oleh admin.
- Task : Admin dapat membuat task yang berisikan judul, deskripsi tugas dan dapat mengupload 1 file sebagai pendukung tugas.
- Task Submission : User dapat mengupload submission tugas (lebih dari 1 file). Pada sisi admin, seorang admin dapat melihat seluruh submission mahasiswa pada sebuah task.

Setup Backend (Express JS)

1. Buatlah sebuah folder dengan nama **Week12_NIMtanpa0** dan didalamnya buatlah folder **backend**. Masuk ke folder **backend** dan install Express JS beserta dependencies lainnya.

```
cd backend
npm init -y
npm install express nodemon multer dotenv cors mysql2 jsonwebtoken bcrypt cookie-parser path
```

2. Setup port server Express JS yang akan digunakan pada file **.env**. Buat file **.env** dan tambahkan **port** yang akan digunakan



3. Buatlah folder **src** dan buatlah file **server.js** di dalam folder **src** tersebut. Salinlah kode dibawah ini:

```

backend > src > JS server.js > ...
1  const express = require("express");
2  const cors = require("cors");
3  const dotenv = require("dotenv");
4  const cookieParser = require("cookie-parser");
5
6  // Load environment variables from .env file
7  dotenv.config();
8
9  const PORT = process.env.PORT;
10
11 const app = express();
12
13 app.use(
14   cors({
15     origin: "http://localhost:5173", // Frontend URL
16     credentials: true, // Allow cookies to be sent
17   })
18 );
19
20 app.use(cookieParser()); // Middleware to parse cookies
21
22 // Middleware to parse JSON requests
23 app.use(express.json());
24
25 app.listen(PORT, () => {
26   console.log(`Server is running on http://localhost:${PORT}`);
27   console.log("Connected to the database successfully");
28 });

```

4. Pada **package.json**, tambahkan kode berikut untuk dapat memulai server Express JS:

```

backend > {} package.json > {} dependencies
1  {
2    "name": "backend",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1",
7      "dev": "nodemon src/server.js"
8    },

```

Database Connection

1. Atur data dari database yang akan digunakan pada file **.env**.

```

backend > .env
Import to Postman
1  PORT=5001
2  NODE_ENV=development
3
4  DB_HOST=localhost
5  DB_USER=root
6  DB_PASSWORD=
7  DB_NAME=AWP_W12

```

2. Pada folder **src**, buatlah file **db.js**. File ini berfungsi untuk melakukan setup database berdasarkan data dari **.env**.

```
backend > src > JS db.js > ...
1  // db.js
2  const mysql = require("mysql2/promise");
3  require("dotenv").config();
4
5  const pool = mysql.createPool({
6    host: process.env.DB_HOST || "localhost",
7    user: process.env.DB_USER || "root",
8    password: process.env.DB_PASS || "",
9    database: process.env.DB_NAME || "AWP_W12"
10 });
11
12 module.exports = pool;
```

3. Pada **server.js**, update kode pada bagian **app.listen** menjadi seperti dibawah ini untuk melakukan koneksi ke database.

```
43 const pool = require("./db");
44
45 // Start server setelah MySQL siap
46 const startServer = async () => {
47   try {
48     const connection = await pool.getConnection();
49     await connection.ping(); // test koneksi
50     connection.release();
51
52     app.listen(PORT, () => {
53       console.log(`Server is running on http://localhost:${PORT}`);
54       console.log("Connected to the database successfully");
55     });
56   } catch (err) {
57     console.error("Unable to connect to the database:", err);
58     process.exit(1); // hentikan kalau DB gagal
59   }
60 };
61
62 startServer();
```

4. Test koneksi database dengan command **npm run dev** pada terminal. Pastikan **XAMPP** telah aktif dan **database telah diimport**.

Authentication

JWT

JWT (JSON Web Token) merupakan token yang dibuat setelah proses login berhasil. JWT merupakan hasil dari encoding (Base64) dan signature dengan secret key yang disimpan di dalam **.env**, membuatnya tidak dapat dipalsukan. JWT juga memiliki masa waktu **expire**. Setelah masa **expire**, maka JWT tidak valid lagi. Token terdiri dari:

1. **Header**: berisi tipe token dan algoritma enkripsi
2. **Payload**: biasanya diisi dengan identity user (id, email, role)
3. **Signature**: Hashing dari header, payload, dan secret key (disimpan dengan aman di **.env**). Signature membuat token tidak dapat dipalsukan tanpa secret key.

Alur kerja autentikasi:

1. Login: user login dengan email dan password.
2. Express akan memverifikasi apakah kombinasi email dan password tersebut benar (sesuai dengan data di database).

3. Jika benar, server akan membuat JWT token dengan masa expire tertentu. JWT token tersebut akan disimpan di cookies dengan konfigurasi **httpOnly** dan **secure** (protokol https).
4. Setiap endpoint yang memerlukan login, client akan mengirim JWT token. Di backend, tambahkan middleware untuk mengecek apakah JWT valid. Middleware ini juga akan menambahkan req.user ke dalam request.

Middleware

1. Pada **.env**, tambahkan JWT_SECRET sebagai secret key untuk hashing signature dari JWT

```
JWT_SECRET=4750e32e7291fa75d52dfb93b5c759e6
```

2. Buat folder **middleware** dan buat file **verifyJWT.middleware.js** di dalamnya. Middleware ini akan mengecek **kevalidan dari JWT** yang dikirim oleh client melalui **req.cookies**. Jika valid, middleware akan mencari user dengan id pada JWT yang telah di decode. Middleware akan menambahkan data **id**, **name**, dan **role** pada **req.user**, berguna untuk controllers mengolah data yang berkaitan dengan user. Tambahkan middleware ini pada setiap **endpoint yang hanya bisa diakses oleh user terautentikasi (telah login)**.

```
backend > src > middleware > JS verifyJWT.middleware.js > ...
1  const jwt = require("jsonwebtoken");
2  const pool = require("../db");
3  require("dotenv").config();
4
5  const verifyJWT = async (req, res, next) => {
6    try {
7      // Client mengirim token di cookie
8      const token = req.cookies?.jwtToken;
9
10     if (!token) {
11       return res.status(401).json({
12         type: "INVALID_TOKEN",
13         message: "Session expired or unauthorized",
14       });
15     }
16
17     // Verifikasi token (sync, kalau gagal akan throw error)
18     let decoded;
19     try {
20       decoded = jwt.verify(token, process.env.JWT_SECRET);
21     } catch (err) {
22       console.error("JWT verify error:", err.message);
23       return res.status(401).json({
24         type: "INVALID_TOKEN",
25         message: "Failed to authenticate token",
26       });
27     }
28
29     // Ambil user dari DB pakai id di token
30     const [rows] = await pool.query(
31       "SELECT id, name, role FROM `Users` WHERE id = ? LIMIT 1",
32       [decoded.id]
33     );
34
35     if (rows.length === 0) {
36       return res.status(404).json({
37         message: "User not found",
38       });
39     }
40
41     req.user = {
42       id: rows[0].id,
43       name: rows[0].name,
44       role: rows[0].role,
45     };
46     next();
47   }
48 }
```

```

41 // Simpan ke req.user untuk dipakai di controller
42 req.user = {
43   id: rows[0].id,
44   name: rows[0].name,
45   role: rows[0].role,
46 };
47
48 next();
49 } catch (error) {
50   console.error("verifyJWT middleware error:", error);
51   return res.status(500).json({
52     message: "Internal server error",
53   });
54 }
55 };
56
57 module.exports = verifyJWT;

```

(backend/src/middleware/verifyJWT.middleware.js)

User

1. Pada **src**, buatlah folder **routes** dan buatlah file **auth.route.js** di dalamnya. Daftarkan route tersebut ke dalam **server.js**

```

const authRouter = require("../routes/auth.route");
app.use("/api/auth", authRouter);

```

2. Pada **src/routes/auth.route.js**, tambahkan endpoint :
 - **/user/create** : Membuat user baru pada database. Hanya admin yang bisa create user.
 - **/login** : Memverifikasi email dan password. Jika valid, maka akan membuat JWT dan menyimpannya ke dalam cookie.
 - **/logout** : Menghapus cookie JWT Refresh Token. Merupakan endpoint yang hanya boleh diakses oleh user yang telah login sehingga perlu middleware verifyJWT.
 - **/me** : Mereturn data user yang sedang login. Merupakan endpoint yang hanya boleh diakses oleh user yang telah login sehingga perlu middleware verifyJWT.

```

backend > src > routes > JS auth.route.js > ...
1  const router = require("express").Router();
2  const verifyJWT = require("../middleware/verifyJWT.middleware");
3
4  const {
5    login,
6    createUser,
7    logout,
8    getMe,
9  } = require("../controllers/auth.controller");
10
11 router.post("/login", login);
12 router.post("/user/create", verifyJWT, createUser);
13 router.get("/logout", logout);
14 router.get("/me", verifyJWT, getMe);
15
16 module.exports = router;

```

Note: Abaikan Error Cannot find module '../controllers/auth.controller'. Hal tersebut dikarenakan kita baru membuat auth.controller di nomor 4.

3. Buatlah folder **utils** dan buatlah file **authValidation.js** di dalamnya. File ini berguna untuk memvalidasi input dari user untuk data autentikasi (register dan login).

```
backend > src > utils > JS authValidation.js > ...
1  const isValidEmail = (email) => {
2    return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
3  };
4  const validateRegistration = (body) => {
5    const errors = [];
6
7    if (!body.email || !body.email.trim()) {
8      errors.push("Email is required");
9    } else if (!isValidEmail(body.email)) {
10     errors.push("Email is not valid");
11   }
12
13   if (!body.name || !body.name.trim()) {
14     errors.push("Name is required");
15   }
16
17   if (!body.role || !["admin", "mahasiswa"].includes(body.role)) {
18     errors.push("Role must be 'admin' or 'mahasiswa'");
19   }
20
21   if (!body.password || body.password.length < 6) {
22     errors.push("Password must be at least 6 characters");
23   }
24
25   return {
26     isValid: errors.length === 0,
27     message: errors.join(", "),
28   };
29 };
30
31 const validateLogin = (body) => {
32   const errors = [];
33
34   if (!body.email || !body.email.trim()) {
35     errors.push("Email is required");
36   } else if (!isValidEmail(body.email)) {
37     errors.push("Email is not valid");
38   }
39
40   if (!body.password) {
41     errors.push("Password is required");
42   }
43
44   return {
45     isValid: errors.length === 0,
46     message: errors.join(", "),
47   };
48 };
49 module.exports = { isValidEmail, validateRegistration, validateLogin };
```

4. Buat folder **controllers** dan buat file **auth.controller.js** di dalamnya.


```

backend > src > controllers > JS auth.controller.js > ...
1  const jwt = require("jsonwebtoken");
2  const bcrypt = require("bcrypt");
3  const { validateRegistration, validateLogin } = require("../utils/authValidation");
4  const pool = require("../db");
5  require("dotenv").config();
6
7  const createUser = async (req, res) => {
8    try {
9      // Validasi data input dari user
10     const { isValid, message } = validateRegistration(req.body);
11     if (!isValid) {
12       return res.status(422).json({ message });
13     }
14
15     const { email, name, role, password } = req.body;
16
17     // Cek apakah email sudah ada
18     const [existing] = await pool.query(
19       "SELECT id FROM `Users` WHERE email = ? LIMIT 1",
20       [email]
21     );
22     if (existing.length > 0) {
23       return res.status(400).json({ message: "Email already exists" });
24     }
25
26     // Hash password dengan bcrypt
27     const hashedPassword = await bcrypt.hash(password, 10);
28
29     // Insert user baru ke dalam database
30     const [result] = await pool.query(
31       `
32       INSERT INTO `Users` (email, name, role, password, createdAt, updatedAt)
33       VALUES (?, ?, ?, ?, NOW(), NOW())
34       `,
35       [email, name, role, hashedPassword]
36     );
37     const newUserId = result.insertId;
38
39     return res.status(201).json({
40       message: "User registered successfully",
41       user: {
42         id: newUserId,
43         email,
44         name,
45         role,
46       },
47     });
48   } catch (error) {
49     console.error("Registration error:", error);
50     return res.status(500).json({ message: "Internal server error" });
51   }
52 };

```



```

54 const login = async (req, res) => {
55   try {
56     // Validasi data input dari user
57     const { isValid, message } = validateLogin(req.body);
58     if (!isValid) {
59       return res.status(422).json({ message });
60     }
61
62     const { email, password } = req.body;
63     // Cari user by email
64     const [rows] = await pool.query(
65       "SELECT id, email, name, role, password FROM `Users` WHERE email = ? LIMIT 1",
66       [email]
67     );
68
69     if (rows.length === 0) {
70       return res.status(401).json({ message: "Invalid email or password" });
71     }
72
73     const user = rows[0];
74
75     // Bandingkan password dari input dengan yang di DB
76     const match = await bcrypt.compare(password, user.password);
77     if (!match) {
78       // Jika tidak cocok, artinya password yang dimasukkan salah
79       return res.status(401).json({ message: "Invalid email or password" });
80     }
81
82     // Buat JWT token berisi id user
83     const jwtToken = jwt.sign(
84       { id: user.id },
85       process.env.JWT_SECRET,
86       { expiresIn: "1h" }
87     );
88
89     // Simpan JWT di cookie
90     res.cookie("jwtToken", jwtToken, {
91       httpOnly: true,
92       secure: process.env.NODE_ENV === "production",
93       sameSite: "lax",
94       maxAge: 60 * 60 * 1000,
95     });
96
97     return res.status(200).json({
98       message: "Login successful",
99       jwtToken,
100       user: {
101         id: user.id,
102         email: user.email,
103         name: user.name,
104         role: user.role,
105       },
106     });
107   } catch (error) {
108     console.error("Login error:", error);
109     return res.status(500).json({ message: "Internal server error" });
110   }
111 };
112
113 const logout = (req, res) => {
114   try {
115     // Mengambil token dari cookie
116     const existingToken = req.cookies?.jwtToken;
117
118     if (!existingToken) {
119       return res.status(400).json({ message: "No active session found" });
120     }
121
122     // Hapus cookie dengan mengatur ulang nilainya
123     res.clearCookie("jwtToken", {
124       httpOnly: true,
125       secure: process.env.NODE_ENV === "production",
126       sameSite: "lax",
127     });
128
129     return res.status(200).json({ message: "Logged out successfully" });
130   } catch (error) {
131     console.error("Logout error:", error);
132     return res.status(500).json({ message: "Internal server error" });
133   }
134 };
135

```

```

136 const getMe = async (req, res) => {
137   try {
138     // req.user diisi middleware verifyJWT
139     const user = req.user;
140
141     if (!user) {
142       return res.status(404).json({ message: "User not found" });
143     }
144
145     return res.status(200).json({
146       user: {
147         id: user.id,
148         email: user.email,
149         name: user.name,
150         role: user.role,
151       },
152     });
153   } catch (error) {
154     console.error("Get user error:", error);
155     return res.status(500).json({ message: "Internal server error" });
156   }
157 };
158
159 module.exports = {
160   createUser,
161   login,
162   logout,
163   getMe,
164 };

```

File Upload

File upload berarti client (browser atau aplikasi) mengirim file ke server melalui HTTP request dengan `enctype="multipart/form-data"` pada form HTML. Browser mengemas file sebagai bagian multipart sehingga server bisa mem-parsing bagian file dan bagian data lainnya.

Di Node.js/Express, kita memerlukan middleware untuk mem-parsing body multipart. Middleware yang akan kita gunakan adalah **multer**.

Task : File Upload

1. Pada **src/routes**, buatlah file **task.route.js**. Tambahkan route tersebut ke **server.js**

```

const taskRouter = require("../routes/task.route");
app.use("/api/task", taskRouter);

```

2. Pada file **task.route.js**. Buat instance dari multer dan konfigurasi destinasi tempat menyimpan file yang diupload, nama file, limit ukuran file, dan jenis file yang diterima. Pada praktikum ini, kita akan menyimpan gambar pada folder **uploads**. Buatlah folder **uploads** pada folder backend (backend/uploads).

```

backend > src > routes > JS task.route.js > ...
1  const router = require("express").Router();
2  const multer = require("multer");
3  const checkRole = require("../middleware/checkRole.middleware");
4  const verifyJWT = require("../middleware/verifyJWT.middleware");
5  const {
6      getAllTasks,
7      getTaskById,
8      createTask,
9      updateTask,
10     deleteTask,
11 } = require("../controllers/task.controller");

backend > src > routes > JS task.route.js > ...
1  const router = require("express").Router();
2  const multer = require("multer");
3  const verifyJWT = require("../middleware/verifyJWT.middleware");
4  const { Task } = require("../models");
5  const {
6      getAllTasks,
7      getTaskById,
8      createTask,
9      updateTask,
10     deleteTask,
11 } = require("../controllers/task.controller");
12
13 // Konfigurasi tempat penyimpanan file
14 const diskStorage = multer.diskStorage({
15     // Menentukan folder penyimpanan file
16     destination: (req, file, cb) => {
17         cb(null, "uploads/");
18     },
19     // Menentukan nama file yang disimpan
20     // Ditambahkan unique suffix agar tidak terjadi duplikasi nama file
21     filename: (req, file, cb) => {
22         const uniqueSuffix =
23             Date.now() + "-" + Math.round(Math.random() * 1e9) + "-";
24         cb(null, uniqueSuffix + file.originalname);
25     },
26 });
27
28 // Multer instance
29 const upload = multer({
30     storage: diskStorage,
31     limits: { fileSize: 5 * 1024 * 1024 }, // Melimit ukuran file hingga 5MB
32
33     // Memfilter file yang diupload agar sesuai dengan tipe yang diizinkan
34     fileFilter: (req, file, cb) => {
35         const allowedTypes = ["image/jpeg", "image/png", "application/pdf"];
36         if (allowedTypes.includes(file.mimetype)) {
37             cb(null, true);
38         } else {
39             cb(new Error("Invalid file type. Only JPEG, PNG, and PDF are allowed."));
40         }
41     },
42 });
43
44 // upload.single("file") digunakan untuk menangani unggahan
45 // file tunggal dengan nama field "file"
46
47 // upload.array("files", 5) digunakan untuk menangani unggahan
48 // beberapa file dengan nama field "files" dan maksimal 5 file
49
50 // upload.fields([ { name: "file", maxCount: 3 }, { name: "image", maxCount: 4 } ])
51 // digunakan untuk menangani unggahan beberapa file dengan nama field yang berbeda
52
53 router.get("/", verifyJWT, getAllTasks);
54 router.get("/:id", verifyJWT, getTaskById);
55 router.post("/", verifyJWT, upload.single("file"), createTask);
56 router.put("/:id", verifyJWT, upload.single("file"), updateTask);
57 router.delete("/:id", verifyJWT, deleteTask);
58
59 module.exports = router;

```

3. Pada **server.js**, tambahkan route `/uploads` yang akan men-serve file statis (image) sesuai dengan nama file yang di request.

Contoh penggunaan :

<http://localhost:5001/uploads/foto1.png>

Server akan memberikan file `foto1.png` yang tersimpan di folder **uploads**.

```

const path = require("path");
// Static files untuk upload
app.use("/uploads", express.static(path.join(__dirname, "..", "uploads")));

```

4. Pada **src/controllers**, buat file **task.controller.js**. Buat logic untuk melakukan CRUD.

```

backend > src > controllers > JS task.controller.js > ...
1 // controllers/task.controller.js
2 const pool = require("../db");
3 const fs = require("fs");
4 const path = require("path");
5
6 // GET /api/task
7 const getAllTasks = async (req, res) => {
8   try {
9     const [tasks] = await pool.query(
10       "SELECT id, title, description, filePath, createdAt, updatedAt FROM `Tasks` ORDER BY createdAt DESC"
11     );
12     return res.status(200).json(tasks);
13   } catch (error) {
14     console.error("Error retrieving tasks:", error);
15     return res.status(500).json({ message: "Error retrieving tasks" });
16   }
17 };
18
19 // GET /api/task/:id
20 const getTaskById = async (req, res) => {
21   try {
22     const { id } = req.params;
23     const user = req.user; // diisi verifyJWT
24
25     // Ambil task
26     const [taskRows] = await pool.query(
27       "SELECT id, title, description, filePath, createdAt, updatedAt FROM `Tasks` WHERE id = ? LIMIT 1",
28       [id]
29     );
30
31     if (taskRows.length === 0) {
32       return res.status(404).json({ message: "Task not found" });
33     }
34
35     const task = taskRows[0];
36
37     // Ambil submissions
38     let submissionsQuery =
39       "SELECT id, taskId, userId, filePath, createdAt, updatedAt FROM `TaskSubmissions` WHERE taskId = ?";
40     const params = [id];
41
42     // Kalau mahasiswa - hanya submission miliknya
43     if (user && user.role === "mahasiswa") {
44       submissionsQuery += " AND userId = ?";
45       params.push(user.id);
46     }
47
48     const [submissions] = await pool.query(submissionsQuery, params);
49
50     return res.status(200).json({
51       ...task,
52       submissions,
53     });
54   } catch (error) {
55     console.error("Error retrieving task:", error);
56     return res.status(500).json({ message: "Error retrieving task" });
57   }
58 };

```

```

60 // POST /api/task
61 const createTask = async (req, res) => {
62   try {
63     const { title, description } = req.body;
64
65     if (!title || !title.trim()) {
66       return res.status(400).json({ message: "Title is required" });
67     }
68
69     const filePath = req.file
70       ? req.file.path.replace(/\\g, "/") // normalisasi windows path
71       : null;
72
73     const now = new Date();
74     const [result] = await pool.query(
75       `
76       INSERT INTO \`Tasks\` (title, description, filePath, createdAt, updatedAt)
77       VALUES (?, ?, ?, ?, ?)
78       `,
79       [title, description || null, filePath, now, now]
80     );
81
82     return res.status(201).json({
83       id: result.insertId,
84       title,
85       description: description || null,
86       filePath,
87       createdAt: now,
88       updatedAt: now,
89     });
90   } catch (error) {
91     console.error("Error creating task:", error);
92     return res.status(500).json({ message: "Error creating task" });
93   }
94 };

```

```

96 // PUT /api/task/:id
97 const updateTask = async (req, res) => {
98   try {
99     const { id } = req.params;
100    const { title, description } = req.body;
101
102    // Cek task apakah exist di database
103    const [rows] = await pool.query(
104      "SELECT id, title, description, filePath FROM `Tasks` WHERE id = ? LIMIT 1",
105      [id]
106    );
107    // Jika tidak ada, maka return error 404
108    if (rows.length === 0) {
109      return res.status(404).json({ message: "Task not found" });
110    }
111
112    const existing = rows[0];
113    // Jika user mengirim title dan description, maka ubah sesuai request body user
114    // Jika tidak ada, maka user tidak ingin mengubah title ataupun description,
115    // maka tidak ada yang perlu diubah
116    let newTitle = title !== undefined && title !== "" ? title : existing.title;
117    let newDescription =
118      description !== undefined ? description : existing.description;
119    let newFilePath = existing.filePath;
120
121    // Kalau ada file baru diupload
122    if (req.file) {
123      const uploadedPath = req.file.path.replace(/\\/g, "/");
124
125      // Hapus file lama kalau ada dan masih exist
126      if (existing.filePath) {
127        try {
128          const oldPath = path.resolve(existing.filePath);
129          if (fs.existsSync(oldPath)) {
130            fs.unlinkSync(oldPath);
131          }
132        } catch (e) {
133          console.warn("Failed to delete old file:", e.message);
134        }
135      }
136      newFilePath = uploadedPath;
137    }
138
139    const now = new Date();
140    await pool.query(
141      `
142      UPDATE `Tasks`
143      SET title = ?, description = ?, filePath = ?, updatedAt = ?
144      WHERE id = ?
145      `,
146      [newTitle, newDescription, newFilePath, now, id]
147    );
148
149    return res.status(200).json({
150      id: Number(id),
151      title: newTitle,
152      description: newDescription,
153      filePath: newFilePath,
154      updatedAt: now,
155    });
156   } catch (error) {
157     console.error("Error updating task:", error);
158     return res.status(500).json({ message: "Error updating task" });
159   }
160 };

```

```

162 // DELETE /api/task/:id
163 const deleteTask = async (req, res) => {
164   try {
165     const { id } = req.params;
166
167     // Cek task
168     const [rows] = await pool.query(
169       "SELECT id, filePath FROM `Tasks` WHERE id = ? LIMIT 1",
170       [id]
171     );
172
173     if (rows.length === 0) {
174       return res.status(404).json({ message: "Task not found" });
175     }
176
177     const task = rows[0];
178
179     // Hapus file kalau ada
180     if (task.filePath) {
181       try {
182         const fileToDelete = path.resolve(task.filePath);
183         if (fs.existsSync(fileToDelete)) {
184           fs.unlinkSync(fileToDelete);
185         }
186       } catch (e) {
187         console.warn("Failed to delete file on task delete:", e.message);
188       }
189     }
190
191     // Hapus semua submission terkait
192     await pool.query("DELETE FROM `TaskSubmissions` WHERE taskId = ?", [id]);
193
194     // Hapus task
195     await pool.query("DELETE FROM `Tasks` WHERE id = ?", [id]);
196
197     return res.status(200).json({ message: "Task deleted successfully" });
198   } catch (error) {
199     console.error("Error deleting task:", error);
200     return res.status(500).json({ message: "Error deleting task" });
201   }
202 };
203
204 module.exports = {
205   getAllTasks,
206   getTaskById,
207   createTask,
208   updateTask,
209   deleteTask,
210 };

```

Authorization

Endpoint yang telah kita buat masih memiliki kekurangan yaitu seluruh user, baik admin ataupun mahasiswa, yang telah login bisa mengakses endpoint yang seharusnya hanya admin yang bisa akses (create user dan CRUD task). Oleh karena itu, perlu ditambahkan otorisasi untuk **memberikan akses hanya kepada role yang berwenang**. Hal ini bisa dilakukan dengan menggunakan **middleware**. Middleware `verifyJWT` telah menambahkan field `req.user` yang berisi data user (id, name, role) yang akan dipakai oleh middleware selanjutnya (`checkRole`) untuk memverifikasi apakah user yang login memiliki role yang diperbolehkan untuk mengakses endpoint tersebut.

1. Buatlah middleware baru **checkRole.middleware.js**. Middleware ini akan menerima arguman berupa role (string) dimana middleware mengecek apakah `req.user.role` sama dengan role yang di allow.


```

backend > src > middleware > JS checkRole.middleware.js > ...
1  const checkRole = (allowedRoles) => {
2      return (req, res, next) => {
3          // req.user sudah diisi oleh middleware verifyJWT
4          const userRole = req.user.role;
5          if (!allowedRoles.includes(userRole)) {
6              return res.status(403).json({
7                  message: "Forbidden: You don't have permissions",
8              });
9          }
10         next();
11     };
12 };
13
14 module.exports = checkRole;

```

2. Update endpoint yang memerlukan role admin untuk mengaksesnya.

```
const checkRole = require("../middleware/checkRole.middleware");
```

```

router.post("/login", login);
router.post("/user/create", verifyJWT, checkRole(["admin"]), createUser);
router.get("/logout", logout);
router.get("/me", verifyJWT, getMe);

```

(backend/src/routes/auth.route.js)

```

router.get("/", verifyJWT, getAllTasks);
router.get("/:id", verifyJWT, getTaskById);
router.post("/", verifyJWT, checkRole(["admin"]), upload.single("file"), createTask);
router.put("/:id", verifyJWT, checkRole(["admin"]), upload.single("file"), updateTask);
router.delete("/:id", verifyJWT, checkRole(["admin"]), deleteTask);

```

(backend/src/routes/task.route.js)

Tugas

Buatlah task submission dimana mahasiswa dapat mengumpulkan jawaban dari tugas yang diberikan dengan ketentuan:

1. Buat fitur CRUD :
 - GET /api/task/submission/:taskId : Jika admin, maka akan mengembalikan seluruh task submission pada task dengan id = taskId. Jika mahasiswa, maka hanya akan mengembalikan task submission miliknya sendiri.
 - POST /api/task/submission/:taskId : Mahasiswa dapat membuat task submission yang berupa file hingga 10 file. File yang diupload di limitasi sampai 7 MB dengan tipe pdf/word/zip
 - PUT /api/task/submission/ : Mahasiswa dapat mengedit task submission yang telah dibuatnya.
 - DELETE /api/task/submission/:taskId : Mahasiswa mendelete seluruh task submission miliknya sesuai dengan task id yang ada di parameter
2. Jangan lupa menerapkan verifyJWT dan checkRole sesuai kebutuhan tiap endpoint.